

Algoritmi di consenso: Raft

Dal problema dell'inconsistenza alla soluzione con log replicato

Last Update: 21 maggio 2026

Il problema: inconsistenza in un sistema replicato

Immaginiamo un conto bancario con un saldo iniziale di 100 euro, replicato su cinque server per garantire che un guasto non causi perdita di dati. Fin qui tutto bene. Il problema emerge nel momento in cui due client inviano operazioni quasi simultaneamente a server diversi.

Supponiamo che il client A e il client B vogliano entrambi prelevare 80 euro dallo stesso conto. Il server 1 riceve la richiesta di A, controlla il saldo (100 euro, prelievo ammesso) e approva. Il server 2 riceve la richiesta di B *nello stesso istante*: anche lui controlla il saldo, vede ancora 100 euro — perché l'aggiornamento del server 1 non è ancora arrivato — e approva.

Risultato: entrambi i prelievi vengono autorizzati. Il cliente A e il cliente B ottengono ciascuno 80 euro, per un totale di 160 euro prelevati da un conto che ne conteneva 100. I cinque server mostrano saldi diversi. Il sistema è **inconsistente**.

❗ La radice del problema

Senza coordinamento, più server possono prendere decisioni incompatibili nello stesso momento, perché ciascuno ragiona sulla propria visione locale dello stato, che può essere già obsoleta.

La causa non è un bug: è strutturale. Ogni server decide autonomamente, senza sapere cosa stanno facendo gli altri in quell'istante. La soluzione non può essere tecnica nel senso di "rendere la rete più veloce". Serve un **meccanismo di coordinamento**.

La soluzione: un arbitro e un log condiviso

L'idea fondamentale di Raft è semplice: invece di lasciare che ogni server decida autonomamente, si elegge uno dei cinque server come **leader**. Tutte le operazioni dei client passano per lui.

Prima di eseguire qualsiasi operazione, il leader la registra in un elenco numerato — il **log** — e lo distribuisce agli altri server. Solo quando la maggioranza dei server ha ricevuto e scritto l'operazione nel proprio log, il leader la esegue e risponde al client.

Tornando all'esempio: quando il client A chiede di prelevare 80 euro, il leader scrive nel log "operazione 1: preleva 80 euro" e la manda agli altri quattro server. Appena tre server su cinque (la maggioranza) hanno confermato di averla ricevuta, il leader esegue il prelievo. Il saldo scende a 20 euro. A questo punto, se arriva la richiesta del client B, il leader controlla il saldo aggiornato — 20 euro — e rifiuta il prelievo di 80 euro.

Il log non è quindi un concetto astratto: è letteralmente l'elenco delle decisioni prese nell'ordine in cui sono state prese. Tutti i server che eseguono le stesse operazioni nello stesso ordine arrivano necessariamente allo stesso stato.

▷ Perché si chiama Raft?

Raft in inglese significa zattera: un insieme di tronchi legati insieme che si muovono come un blocco unico. I server in Raft fanno la stessa cosa — si muovono tutti insieme, coordinati, come se fossero un sistema unico invece di cinque macchine separate.

Visione d'insieme dell'algoritmo

Raft si articola in tre meccanismi distinti che cooperano tra loro.

Elezione del leader. Il sistema parte senza un leader. Uno dei server si candida (stato *candidate*) per un mandato, raccoglie i voti della maggioranza ed eventualmente diventa leader. Da quel momento in poi è l'unico a prendere decisioni. Se il leader crasha, i follower se ne accorgono dalla mancanza di heartbeat e avviano una nuova elezione.

Replicazione del log. Tutte le operazioni dei client arrivano al leader. Il leader le aggiunge al proprio log e le replica sui follower. Solo quando la maggioranza ha confermato la ricezione, l'operazione diventa definitiva (*committed*) e viene eseguita. I follower eseguono le operazioni nello stesso ordine del leader.

Sicurezza dopo un crash. Quando viene eletto un nuovo leader, il suo log potrebbe non essere identico a quello di tutti i follower. Raft risolve questo in modo asimmetrico: i follower si allineano sempre al leader, mai il contrario. Il leader trova il punto di divergenza e sovrascrive le entry inconsistenti sui follower.



I tre meccanismi formano un ciclo: l'elezione produce un leader, il leader replica il log sui server, se il leader crasha si avvia una nuova elezione. Il sistema non si ferma mai, finché la maggioranza dei server è raggiungibile.

I messaggi scambiati

Raft usa solo due tipi di messaggi. Ogni altro aspetto del protocollo — heartbeat, richieste di voto, replicazione, riconciliazione del log dopo un crash — è implementato come caso particolare di questi due.

RequestVote — richiesta di voto

Inviato da un **candidate** a tutti gli altri server durante un'elezione.

Campo	Tipo	Significato
<code>term</code>	intero	Il term per cui il candidato si propone
<code>candidateId</code>	intero	Identificatore del candidato
<code>lastLogIndex</code>	intero	Indice dell'ultima entry nel log del candidato
<code>lastLogTerm</code>	intero	Term dell'ultima entry nel log del candidato

Risposta: ogni server risponde con il proprio `currentTerm` e un booleano `voteGranted`. Il voto viene concesso solo se il candidato ha un `term` $\geq$ al proprio e il suo log è almeno

aggiornato quanto quello del votante. Viene memorizzato il voto, in modo tale che all'interno di un term ogni server voti per una volta sola.

AppendEntries — replica del log e heartbeat

Inviato dal **leader** a tutti i follower. Svolge due funzioni: con **entries** non vuoto replica nuove operazioni; con **entries** vuoto funziona da heartbeat, segnalando che il leader è vivo.

Campo	Tipo	Significato
term	intero	Term corrente del leader
leaderId	intero	Identificatore del leader
prevLogIndex	intero	Indice dell'entry precedente alle nuove
prevLogTerm	intero	Term di quella entry
entries[]	array	Le nuove entry da aggiungere (vuoto = heartbeat)
leaderCommit	intero	Indice dell'ultima entry committed dal leader

Risposta: il follower risponde con il proprio **currentTerm** e un booleano **success**.

Come funziona Raft: descrizione generale

Ruoli e struttura del tempo

In ogni momento ciascuno dei server si trova in uno di tre stati: **leader**, **follower** o **candidate**. Normalmente c'è un solo leader e tutti gli altri sono follower. I candidate esistono solo durante una transizione, quando il leader corrente è crashato e bisogna eleggerne uno nuovo.

Il tempo in Raft è diviso in **term** — mandati, numerati in modo progressivo. Ogni term inizia con un'elezione e ha al massimo un leader.

L'elezione del leader

Il leader manda periodicamente **AppendEntries** vuoti a tutti i follower come heartbeat. Se un follower non ne riceve uno entro il timeout, presume che il leader sia morto e si candida: incrementa il proprio term, vota per sé stesso e manda **RequestVote** a tutti.

Un server concede il voto solo se il log del candidato è **almeno aggiornato quanto il proprio**. Il confronto avviene in due passi: prima si confronta il term dell'ultima entry — chi ha il term più alto vince. Se i term sono uguali, vince chi ha il log con più entries.

Se due candidati si presentano simultaneamente (caso rarissimo) e nessuno raggiunge la maggioranza, si aspetta un timeout *casuale* prima di riprovare. Il timeout casuale evita che i candidati si blocchino a vicenda all'infinito: statisticamente, uno dei due si sveglia prima e vince (vince chi arriva primo).

La replicazione del log

Quando il leader riceve un'operazione da un client, la aggiunge al proprio log come entry **pendente**. La manda poi in parallelo a tutti i follower tramite **AppendEntries**. Quando la maggioranza ha risposto con successo, l'entry diventa **committed**. Il leader la esegue e risponde al client.

La consistenza dopo un crash

Quando viene eletto un nuovo leader, il suo log potrebbe divergere da quello di alcuni follower - Raft risolve questo con una regola semplice: i follower si allineano sempre al leader. Il leader

manderà dei nuov **AppendEntries** che sovrascriveranno il log locale dei follower eliminando l'inconsistenza